

# JC4004 – Computational Intelligence

## Sessions 13 & 14: Swarm intelligence

Dr Yongchao Huang ([yongchao.huang@abdn.ac.uk](mailto:yongchao.huang@abdn.ac.uk))

Dr Shahzad Mumtaz ([shahszad.mumtaz@abdn.ac.uk](mailto:shahszad.mumtaz@abdn.ac.uk))

Dr Jari Korhonen ([jari.korhonen@abdn.ac.uk](mailto:jari.korhonen@abdn.ac.uk))

Oct 2025

# What is swarm intelligence?

- *Swarm intelligence* is based on the idea that autonomous agents (or particles) can solve complex tasks together by following simple rules
  - Swarm intelligence is inspired by behaviour of groups of animals in nature (e.g., flocks of birds or colonies of ants)
  - Swarm intelligence is *decentralised*: agents decide their actions independently based on their interaction with the environment
- Swarm intelligence has several applications
  - Solving practical problems like routing in telecommunications systems
  - Simulating complex systems like social networks or economic markets

# Background

- We can observe coordinated and meaningful navigation of groups of animals (from tens to thousands) in many animal societies
  - Each individual only uses information about the immediate surroundings
  - There can be simple communication between the individuals
  - Usually, there is no predefined group leader

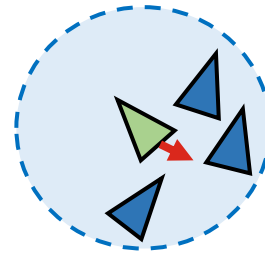
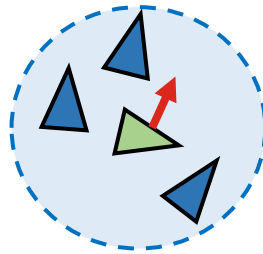
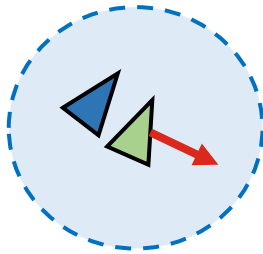


# Basic principles

- Swarm intelligence is the collective intelligence of groups of simple individuals (swarms), based on three main principles:
  - 1) The swarm can solve complex problems that a single individual with simple abilities could not solve
  - 2) The performance of the swarm is not affected by loss of one individual or a mistake by one individual
  - 3) Individuals have local sensory information and they can perform simple actions, but they have little or no memory, and they do not know about the status of the swarm as a whole or its goals

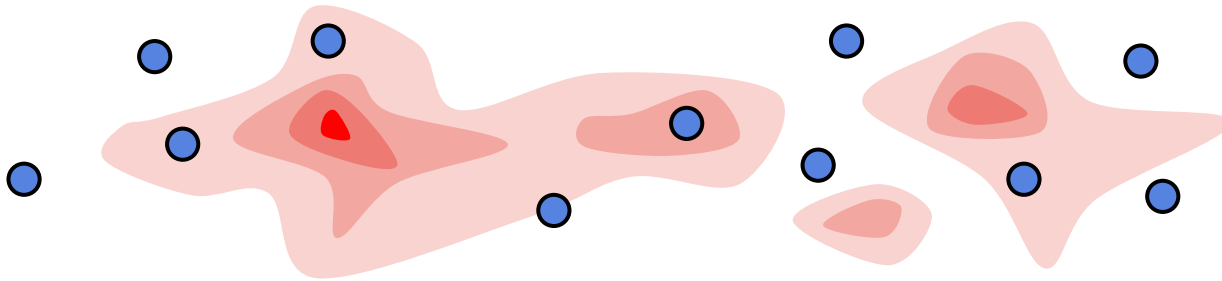
# Reynold's boids

- **Craig Reynolds** defined "bird-oid objects" (boids) to simulate flocking behaviour of birds in a computer program in 1986
- The boids follow three simple rules:
  - *Separation*: steer to avoid getting too close to the local neighbour boids
  - *Alignment*: steer towards the average direction of the local neighbours
  - *Cohesion*: steer to move towards the centre of mass of the local neighbours



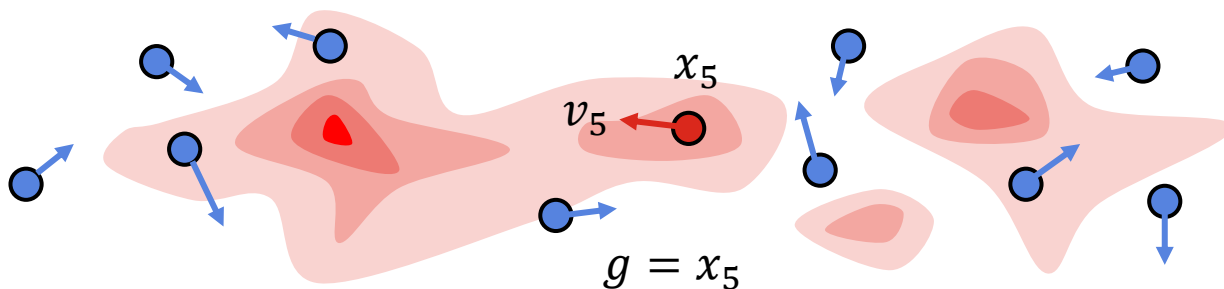
# Particle swarm optimisation (PSO)

- PSO is an optimisation algorithm inspired by birds flocking to find food
  - The flock wants to find the area of the highest concentration of insects
  - The birds can signalise their position and the food concentration there to their neighbours
  - The birds can remember their past position of the best food concentration



# PSO initialisation

- Each particle  $i$  is described by its current position  $x_i$ , best known position  $p_i$ , velocity  $v_i$ , and cost function values  $f(x_i)$  and  $f(p_i)$ 
  - We want to find the position that minimises the cost function
  - Positions and velocities of the particles are initialised randomly, initial  $p_i$  is set to  $x_i$
  - The swarms best known position  $g$  is set to the  $x_i$  that minimises  $f(g)$

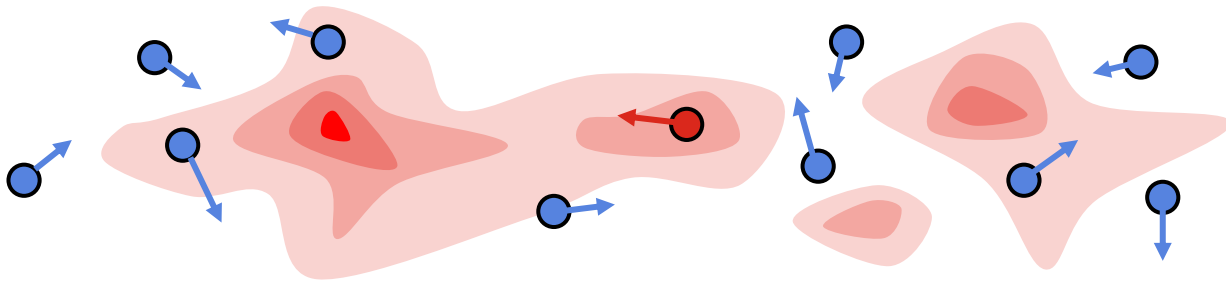


# PSO velocity update

- In each step, random values  $r_p, r_g \sim \mathcal{U}(0,1)$  are chosen for each particle, and the velocities are updated as:

$$v_i \leftarrow wv_i + \varphi_p r_p (p_i - x_i) + \varphi_g r_g (g - x_i)$$

- For velocities and positions with multiple dimensions, we need to repeat the process for each dimension
- Parameters  $w$ ,  $\varphi_p$ , and  $\varphi_g$  are set by the controller of the algorithm





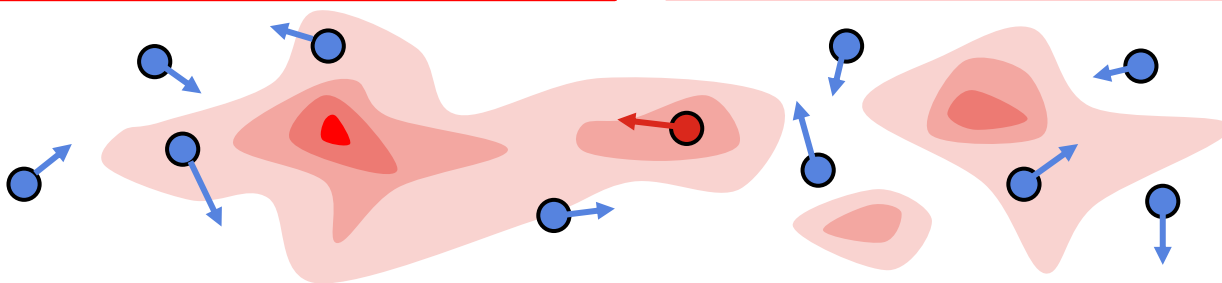
# PSO velocity update

- In each step, random values  $r_p, r_g \sim \mathcal{U}(0,1)$  are chosen for each particle, and the velocities are updated as:

$$v_i \leftarrow wv_i + \underbrace{\varphi_p r_p (p_i - x_i)}_{\text{steers towards past best}} + \underbrace{\varphi_g r_g (g - x_i)}_{\text{steers towards global best}}$$

This term steers the particle towards its past best position

This term steers the particle towards the global best position

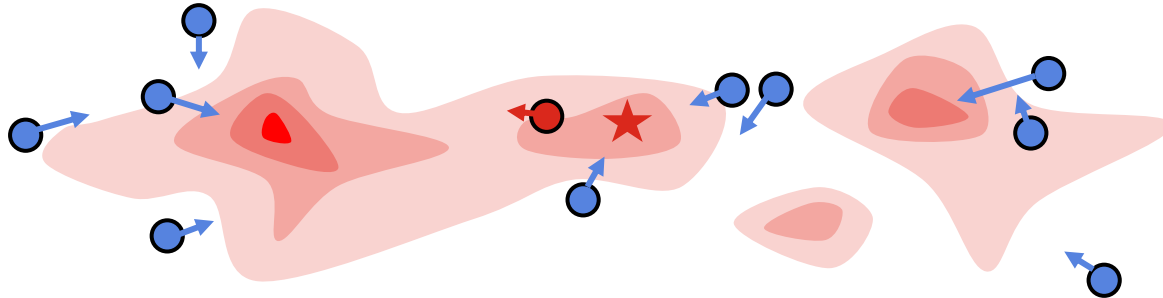


# PSO position update

- In each step, the positions of all the particles are update using their new velocities:

$$x_i \leftarrow x_i + v_i$$

- If new best position for the particle is found,  $p_i$  is updated
- If new global best position is found, value for  $g$  is updated



# PSO termination

- If the parameters are selected appropriately, all the particles are eventually expected to approach the global optimum
  - Termination criterion can be e.g., the number of iterations, or achieving a threshold value for the cost function in the global best position
  - If termination criteria not fulfilled, the process is repeated and the velocities are updated again



# PSO Python example: cost

```
1  import random
2  import numpy as np
3
4  mu_x = []; mu_y = []; sigma_x = []; sigma_y = []; amp = []
5  for i in range(0, random.randint(5,10)):
6      mu_x.append(random.uniform(-80,80))
7      mu_y.append(random.uniform(-80,80))
8      sigma_x.append(random.uniform(1000.0,10000.0))
9      sigma_y.append(random.uniform(1000.0,10000.0))
10     amp.append(random.uniform(-50.0, -10.0))
11
12 def cost(x,y):
13     tot_cost = 0
14     for i in range(0, len(mu_x)):
15         tot_cost += amp[i]*np.exp(-(((x-mu_x[i])**2)/sigma_x[i]
16                                     + ((y-mu_y[i])**2)/sigma_y[i])))
17     return tot_cost
```

# PSO Python example: cost

```
1 import random
2 import numpy as np
3
4 mu_x = []; mu_y = []; sigma_x = []; sigma_y = []
5 for i in range(0, random.randint(5, 10)):
6     mu_x.append(random.uniform(-80, 80))
7     mu_y.append(random.uniform(-80, 80))
8     sigma_x.append(random.uniform(1000.0, 10000.0))
9     sigma_y.append(random.uniform(1000.0, 10000.0))
10    amp.append(random.uniform(-50.0, -10.0))
11
12 def cost(x, y):
13     tot_cost = 0
14     for i in range(0, len(mu_x)):
15         tot_cost += amp[i] * np.exp(-(((x - mu_x[i])**2)/sigma_x[i]
16                                         + ((y - mu_y[i])**2)/sigma_y[i])))
17     return tot_cost
```

Cost function uses randomly generated Gaussian-shaped “dips” to form a 2D landscape used as cost function

# PSO Python example: initialisation

```
18 x_x = []; x_y = []; v_x = []; v_y = []
19 min_cost = []
20 min_cost_g = float("inf")
21 g_x = 0
22 g_y = 0
23 for i in range(0,5):
24     x_x.append(random.uniform(-50,50))
25     x_y.append(random.uniform(-50,50))
26     v_x.append(random.uniform(-1,1))
27     v_y.append(random.uniform(-1,1))
28     min_cost.append(cost(x_x[i],x_y[i]))
29     if min_cost[i] < min_cost_g:
30         min_cost_g = min_cost[i]
31         g_x = x_x[i]; g_y = x_y[i]
32
33 p_x = x_x.copy(); p_y = x_y.copy()
34 w = 0.5; fi_p = 0.02; fi_g = 0.3
```

# PSO Python example: initialisation

```
18 x_x = []; x_y = []; v_x = []; v_y = []
19 min_cost = []
20 min_cost_g = float("inf")
21 g_x = 0
22 g_y = 0
23 for i in range(0,5):
24     x_x.append(random.uniform(-50,50))
25     x_y.append(random.uniform(-50,50))
26     v_x.append(random.uniform(-1,1))
27     v_y.append(random.uniform(-1,1))
28     min_cost.append(cost(x_x[i],x_y[i]))
29     if min_cost[i] < min_cost_g:
30         min_cost_g = min_cost[i]
31         g_x = x_x[i]; g_y = x_y[i]
32
33 p_x = x_x.copy(); p_y = x_y.copy()
34 w = 0.5; fi_p = 0.02; fi_g = 0.3
```

Generate random  
positions for particles

Generate random  
velocities for particles

Test if global optimum

Parameters

# PSO Python example: update

```
35 def update(g_x,g_y,min_cost_g):
36     for i in range(0,len(x_x)):
37         x_x[i] += v_x[i]; x_y[i] += v_y[i]
38         v_x[i] = w*v_x[i] + random.uniform(0,1)*fi_p*(p_x[i]-x_x[i])
39         v_x[i] += random.uniform(0,1)*fi_g*(g_x-x_x[i])
40         v_y[i] = w*v_y[i] + random.uniform(0,1)*fi_p*(p_y[i]-x_y[i])
41         v_y[i] += random.uniform(0,1)*fi_g*(g_y-x_y[i])
42         if cost(x_x[i],x_y[i]) < min_cost[i]:
43             min_cost[i]=cost(x_x[i],x_y[i]); p_x[i]=x_x[i]; p_y[i]=x_y[i]
44         if min_cost[i] < min_cost_g:
45             min_cost_g = min_cost[i]; g_x = x_x[i]; g_y = x_y[i]
46     return g_x,g_y,min_cost_g
47
48 for i in range(0,20):
49     g_x,g_y,min_cost_g = update(g_x,g_y,min_cost_g)
50     for j in range(0,5):
51         print(x_x[j],x_y[j],min_cost[j])
52     print('----')
```



# PSO Python example: update

```
35 def update(g_x,g_y,min_cost_g):
36     for i in range(0,len(x_x)):
37         x_x[i] += v_x[i]; x_y[i] += v_y[i]
38         v_x[i] = w*v_x[i] + random.uniform(0,1)*fi_p*(p_x[i]-x_x[i])
39         v_x[i] += random.uniform(0,1)*fi_g*(g_x-x_x[i])
40         v_y[i] = w*v_y[i] + random.uniform(0,1)*fi_p*(p_y[i]-x_y[i])
41         v_y[i] += random.uniform(0,1)*fi_g*(g_y-x_y[i])
42         if cost(x_x[i],x_y[i]) < min_cost[i]:
43             min_cost[i]=cost(x_x[i],x_y[i]); p_x[i]=x_x[i]; p_y[i]=x_y[i]
44             if min_cost[i] < min_cost_g:
45                 min_cost_g = min_cost[i]; g_x = x_x[i]; g_y = x_y[i]
46     return g_x,g_y,min_cost_g
```

Update positions

Update velocities

Test if individual or global minimum reached

```
47
48     e(g_x,g_y,min_cost_g)
49     print('-----')
50     print(x_x[j],x_y[j],min_cost[j])
51
52     print('-----')
```

# PSO Python example: update

```
35 def update(g_x,g_y,min_cost_g):
36     for i in range(0,len(x_x)):
37         x_x[i] += v_x[i]; x_y[i] += v_y[i]
38         v_x[i] = w*v_x[i] + random.uniform(0,1)*fi_p*(p_x[i]-x_x[i])
39         v_x[i] += random.uniform(0,1)*fi_g*(g_x-x_x[i])
40         v_y[i] = w*v_y[i] + random.uniform(0,1)*fi_p*(p_y[i]-x_y[i])
41         v_y[i] += random.uniform(0,1)*fi_g*(g_y-x_y[i])
42         if cost(x_x[i],x_y[i]) < min_cost[i]:
43             min_cost[i]=cost(x_x[i],x_y[i]); p_x[i]=x_x[i]; p_y[i]=x_y[i]
44         if min_cost[i] < min_cost_g:
45             min_cost_g = min_cost[i]; g_x = x_x[i]; g_y = x_y[i]
46     return g_x,g_y,min_cost_g
47
48 for i in range(0,20):
49     g_x,g_y,min_cost_g = update(g_x,g_y,min_cost_g)
50     for j in range(0,5):
51         print(x_x[j],x_y[j],min_cost[j])
52     print('----')
```

Main loop

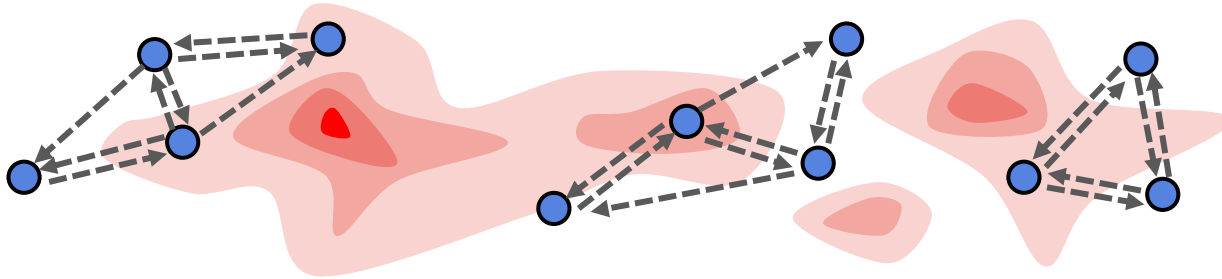
# PSO Python example: test run

```
35 def update(g_x,g_y,min_cost_g):
36     for i in range(0,len(x x)):
37         x_x[i] += -14.459269251593943 -19.628181022326558 -121.73249046431188
38         v_x[i] = 26.040958966002705 -6.036630891663468 -110.23824158332295
39         v_x[i] += -14.224056331187757 -28.657967706999095 -125.87408345822692
40         v_y[i] = 40.088460306191294 42.01676468658926 -54.26704777759527
41         v_y[i] += -27.188258264345272 -36.137655248169146 -123.50867063805376
42         if cost(x_x[i],x_y[i]) < min_cost_g[i]:
43             min_cost_g[i] = cost(x_x[i],x_y[i])
44             if min_cost_g[i] < min_cost_g[0]:
45                 min_cost_g[0] = min_cost_g[i]
46     return g_x,g_y,min_cost_g
47
48 for i in range(0,len(x x)):
49     g_x,g_y,min_cost_g = update(g_x,g_y,min_cost_g)
50     for j in range(0,len(x x)):
51         print(x_x[j],x_y[j],min_cost_g[j])
52     print('----')
```

**Break: any questions?**

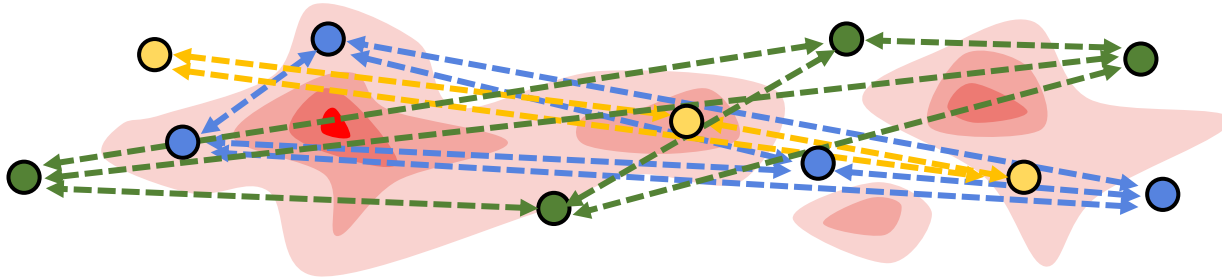
# PSO neighbourhoods and topologies

- The baseline version of PSO uses global topology, i.e., all the particles know the global best position
  - This approach can lead the swarm to be trapped to a local minimum
- Particles could also share information only with a subset of particles
  - *Geometrical subset*, a specific number of the nearest particles
  - *Social subset*, sets of particles allocated in the initialisation



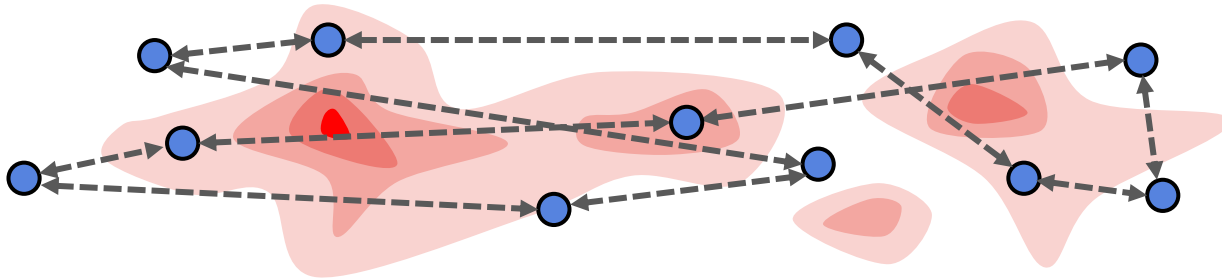
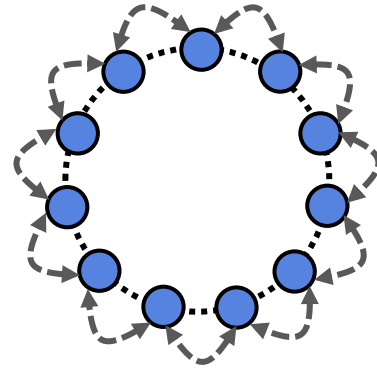
# PSO neighbourhoods and topologies

- The baseline version of PSO uses global topology, i.e., all the particles know the global best position
  - This approach can lead the swarm to be trapped to a local minimum
- Particles could also share information only with a subset of particles
  - *Geometrical subset*, a specific number of the nearest particles
  - *Social subset*, sets of particles allocated in the initialisation



# PSO ring topology

- *Ring topology* is commonly used for PSO
  - Particles allocated to a ring in initialisation
  - Each particle communicates only with its two neighbours
  - The topology can be static or dynamic



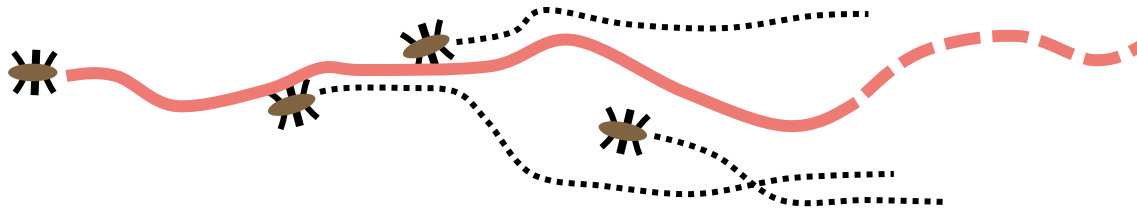
# Notes about PSO

- PSO is based on metaheuristics and makes few or no assumptions about the problem to be solved
  - The loss function does not need to be differentiable like in optimisation methods based on gradient descend
  - However, *there is no guarantee that the optimal solution will be found!*
- Several different variants of PSO have been developed
  - Different ways for initialisation, setting constraints to the velocities, multi-objective optimisation, etc.
  - Hybridisation: combination of PSO with other optimisation methods



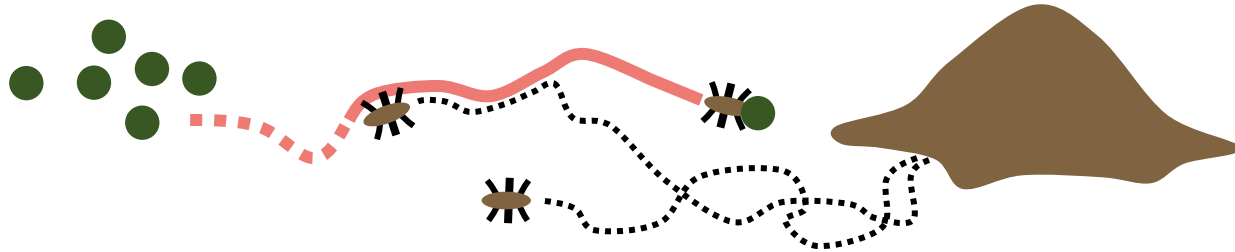
# Stigmerky

- *Stigmergy* indicates communication between individuals through modification of the environment
  - The individuals in a swarm cannot communicate with each other directly, but they can leave signs that the others can observe
  - For example, some ants can release a chemical trail of pheromones the other ants will follow; *however, the trail decays over time*
  - Idea of stigmerky is used in e.g., ant colony optimisation



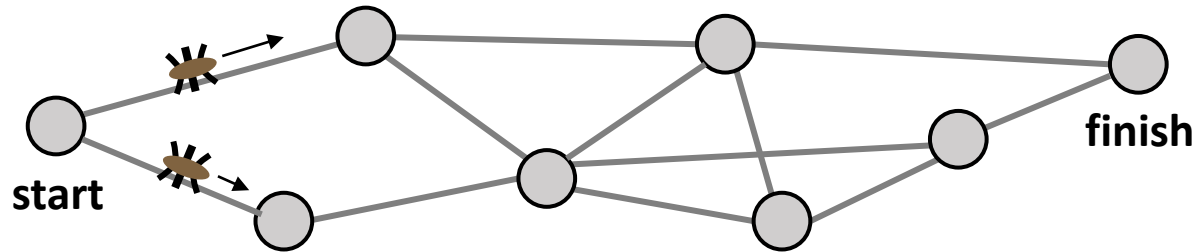
# Ant colony optimisation (ACO)

- ACO algorithm is inspired by ants looking for food and returning to their colonies
  - Initially, ants wander randomly looking for food
  - Upon finding food, they return to their colony while leaving a pheromone trail to mark their route
  - Other ants are likely to follow the pheromone trail and reinforce it in case they also find food



# ACO algorithm

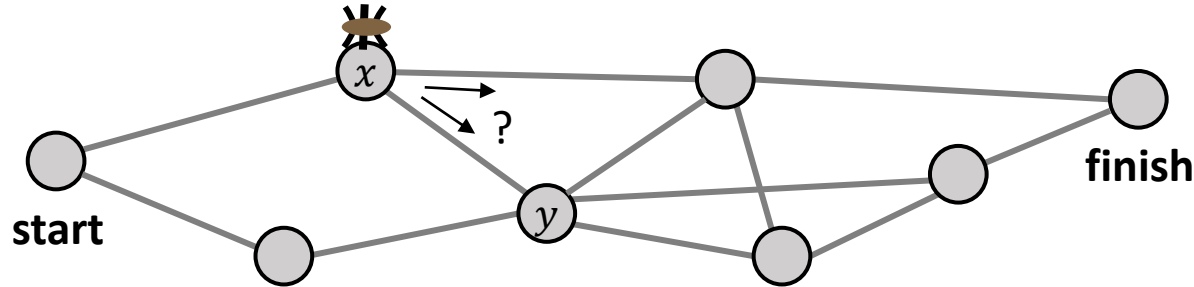
- To use ACO algorithm, optimisation problem needs to be defined as a problem of finding the shortest path on a weighted graph
  - In the first step of each iteration, each ant finds a path stochastically
  - In the second step, different paths found are compared
  - In the third step, pheromone levels of each edge are updated



# ACO transition rule

- Each edge from node  $x$  to  $y$  is characterised by their *attractiveness*  $\eta(x, y)$  and *pheromone level*  $\tau(x, y)$  initialised to  $\tau_0$ 
  - Ant  $k$  chooses the edge randomly, following the probability distribution:

$$p_k(x, y) = \frac{\tau(x, y)^\alpha \cdot \eta(x, y)^\beta}{\sum_{z \in S} \tau(x, z)^\alpha \cdot \eta(x, z)^\beta}$$

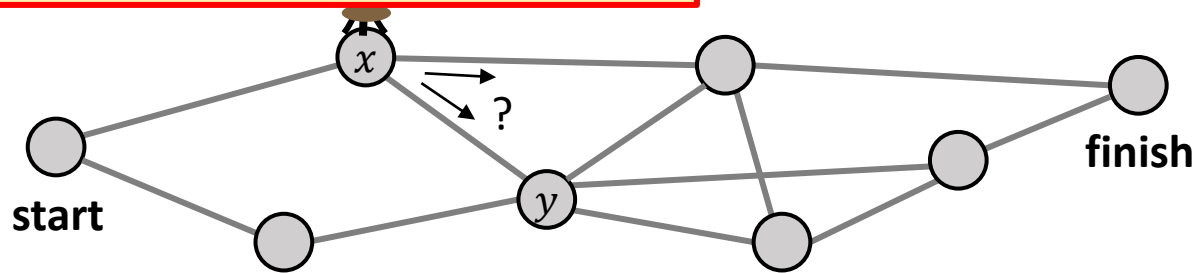


# ACO transition rule

- Each edge from node  $x$  to  $y$  is characterised by their *attractiveness*  $\eta(x, y)$  and *pheromone level*  $\tau(x, y)$  initialised to  $\tau_0$
- Ant  $k$  chooses the edge randomly, following the probability distribution:

Typically,  $\eta(x, y) = 1/L_{x,y}$ , where  $L_{x,y}$  is the length of edge from  $x$  to  $y$

$$\frac{\eta(x, y)^\beta}{\sum_z \tau(x, z)^\alpha \cdot \eta(x, z)^\beta}$$

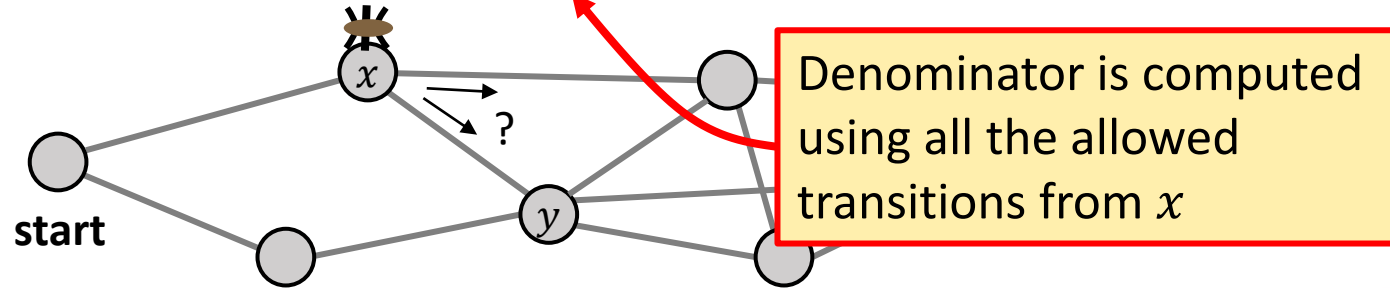


# ACO transition rule

- Each edge from node  $x$  to node  $y$  is characterized by its attractiveness  $\eta(x, y)$  and *pheromone*  $\tau(x, y)$ 
  - Ant  $k$  chooses the edge  $x \rightarrow y$  with probability distribution:

Parameters  $\alpha$  and  $\beta$  define the relative weights of  $\eta$  and  $\tau$

$$p_k(x, y) = \frac{\tau(x, y)^\alpha \cdot \eta(x, y)^\beta}{\sum_{z \in S} \tau(x, z)^\alpha \cdot \eta(x, z)^\beta}$$

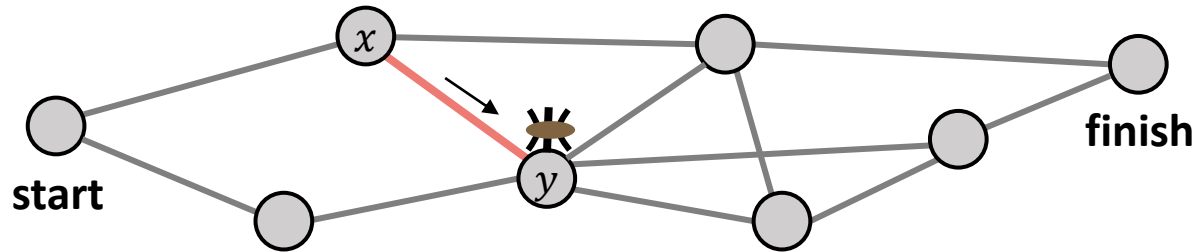


# Local pheromone update

- Pheromones can be updated *locally* every time after an ant visits an edge, or *globally* after all the ants of the batch have found their routes
  - Pheromones for edge  $x$  to  $y$  updated locally after an ant visits:

$$\tau(x, y) \leftarrow (1 - \rho) \tau(x, y) + \rho \tau_0$$

- Parameter  $\rho$  is *pheromone evaporation coefficient*

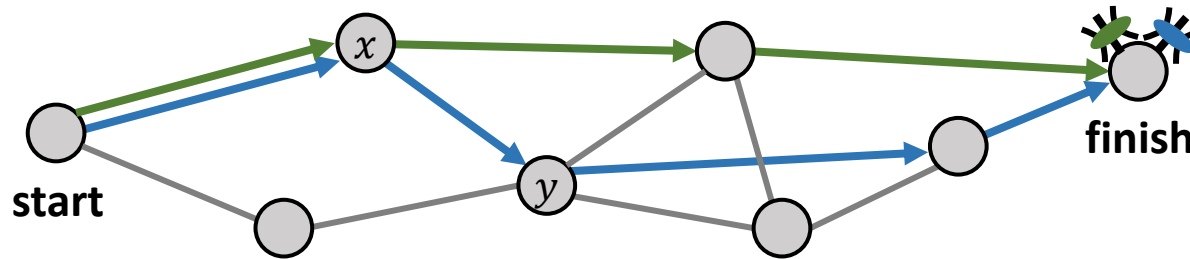


# Global pheromone update

- In global update, the pheromone levels for the edges are updated after all the ants in the batch have completed their journey
  - An example rule for global update for transition from  $x$  to  $y$ :

$$\tau(x, y) \leftarrow (1 - \rho) \tau(x, y) + \sum_{k=1}^m \Delta\tau_k(x, y)$$

- $\Delta\tau_k(x, y)$  is the amount of pheromone deposit by ant  $k$  of  $m$  ants



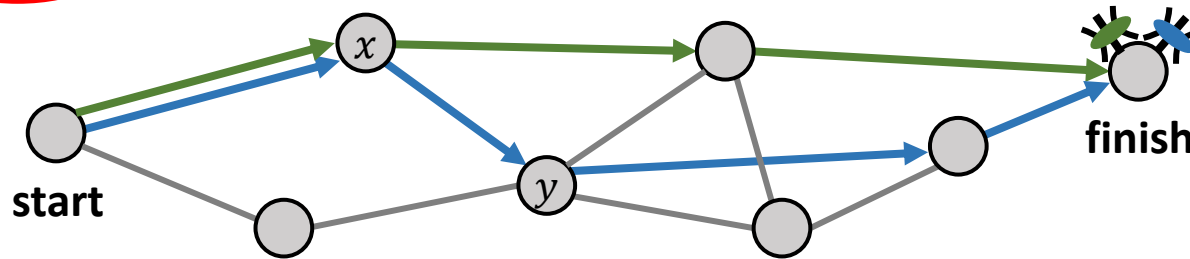


# Global pheromone update

- $\Delta\tau_k(x, y) = 0$ , if ant  $k$  did not use transition from  $x$  to  $y$ ; d after  
 $\Delta\tau_k(x, y) = Q/L_k$ , if ant  $k$  used transition from  $x$  to  $y$ :  $Q$  is a constant, and  $L_k$  is the total cost (length) of the route by ant  $k$

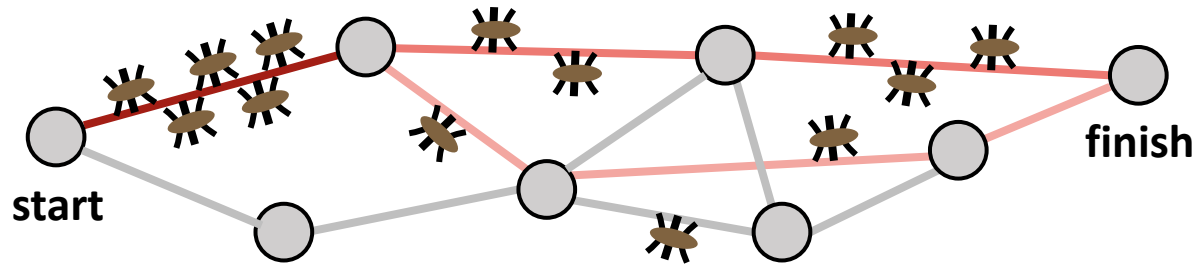
$$\tau(x, y) \leftarrow (1 - \rho) \tau(x, y) + \sum_{k=1}^m \Delta\tau_k(x, y)$$

- $\Delta\tau_k(x, y)$  is the amount of pheromone deposit by ant  $k$  of  $m$  ants



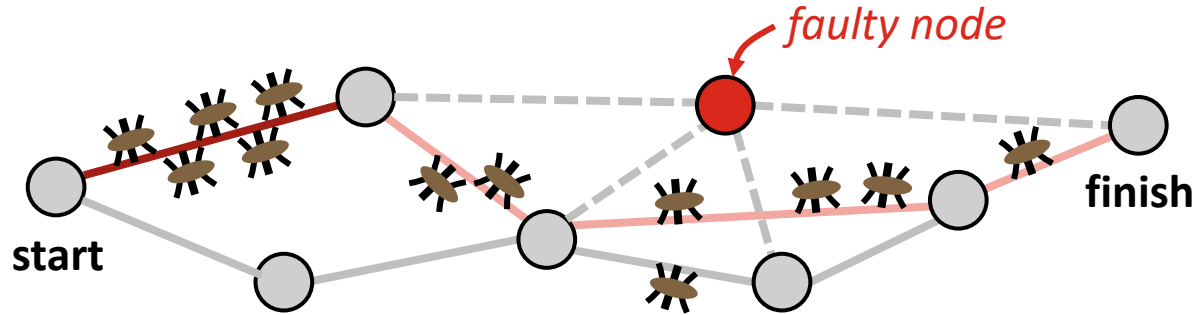
# ACO iteration

- After updating the pheromones, new batch of ants can be released to journey the graph
  - Ants most likely follow routes with high pheromone levels, but due to the random element in route selection, they can also find new routes



# Dynamic routing

- ACO algorithm can also be used for dynamic rerouting in situations where the original best path is not available any more
  - For example, recovering a faulty node in a telecom network
  - ACO algorithm finds the new shortest path quickly: variants of the algorithm used by many telecom operators



# Artificial immune system (AIS)

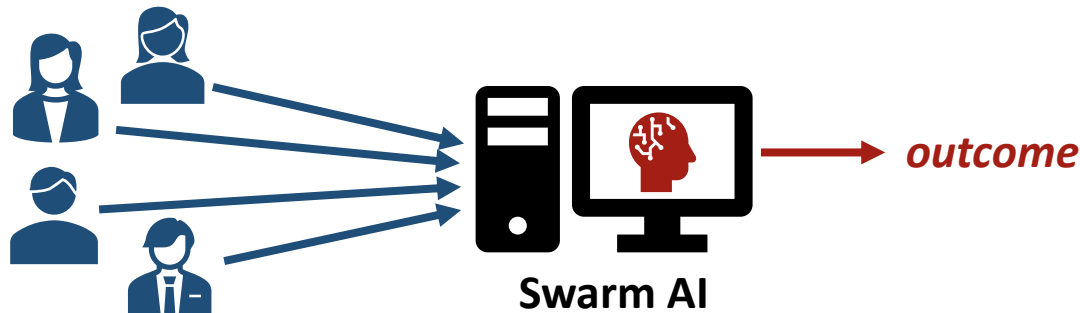
- AIS is a class of algorithms inspired by the principles and processes of the immune system in animals
  - Several subclasses of algorithms, such as *clonal selection*, *negative selection*, *immune network*, and *dendritic cell* algorithms
  - Technically, AIS does not belong to swarm intelligence; however, AIS and swarm intelligence are closely related concepts sharing many similarities
- AIS algorithms have been used in a wide range of applications
  - For example, immune network algorithms have been used for recommendation systems
  - However, note that AIS is *not* for studying biological immune systems!

# Challenges in swarm intelligence

- The main challenge in swarm intelligence is to find the rules for the individual that result to the desired swarm intelligence
  - Algorithms like PSO and ACO are rarely applicable to real problems in their basic form: different variants of the methods are required
  - Fortunately, the rules are supposed to be simple
  - The rules are usually manually designed, but could also be evolved
- Another challenge is to make the emergent behaviour stable
  - Swarm behaviour can be difficult to predict before trying it in practice
  - Algorithm performance depends on the problem and data

# Artificial swarm intelligence (ASI)

- The classic swarm intelligence algorithms are inspired by animals with limited cognitive reasoning skills
  - However, the idea of collective intelligence could also be applied to networked groups of humans solving very complex problems
  - ASI or *human swarming* refers to AI algorithms that can amplify collective intelligence of the humans interacting with the system



# Summary

- *Swarm intelligence* refers to a family of biologically inspired techniques for solving complex problems through collaboration
  - The individuals follow simple rules, and they have limited means of communication and observation
  - The swarm can solve a problem the individuals could not solve alone
- Several different algorithms belong to the area of swarm intelligence
  - Baseline algorithms for *Particle swarm optimisation (PSO)* and *Ant colony optimisation (ACO)* introduced as example of swarm intelligence
  - Other topics related to swarm intelligence include e.g., *Artificial Immune System (AIS)* and *Artificial Swarm Intelligence (ASI)*

# Questions and discussion